ASSIGNMENT 4 — FINAL

Personal Portfolio *Web Application*

I build. I learn. I iterate.

Ibrahim Alshaya

Software Engineering · KFUPM

HTML5 | CSS3 | JavaScript | GitHub API

INTRODUCTION

My portfolio that had to earn its keep.

Personal motivation

I'm applying for AI and data engineering internships this cycle. A CV lists the work and a portfolio shows it. I wanted one site that loads my real projects, pulls my live GitHub activity, and doesn't feel like a template.

PROJECT OBJECTIVES

Ship a production-quality static site

Vanilla HTML, CSS, JavaScript - no framework shortcuts.

Integrate a live third-party API

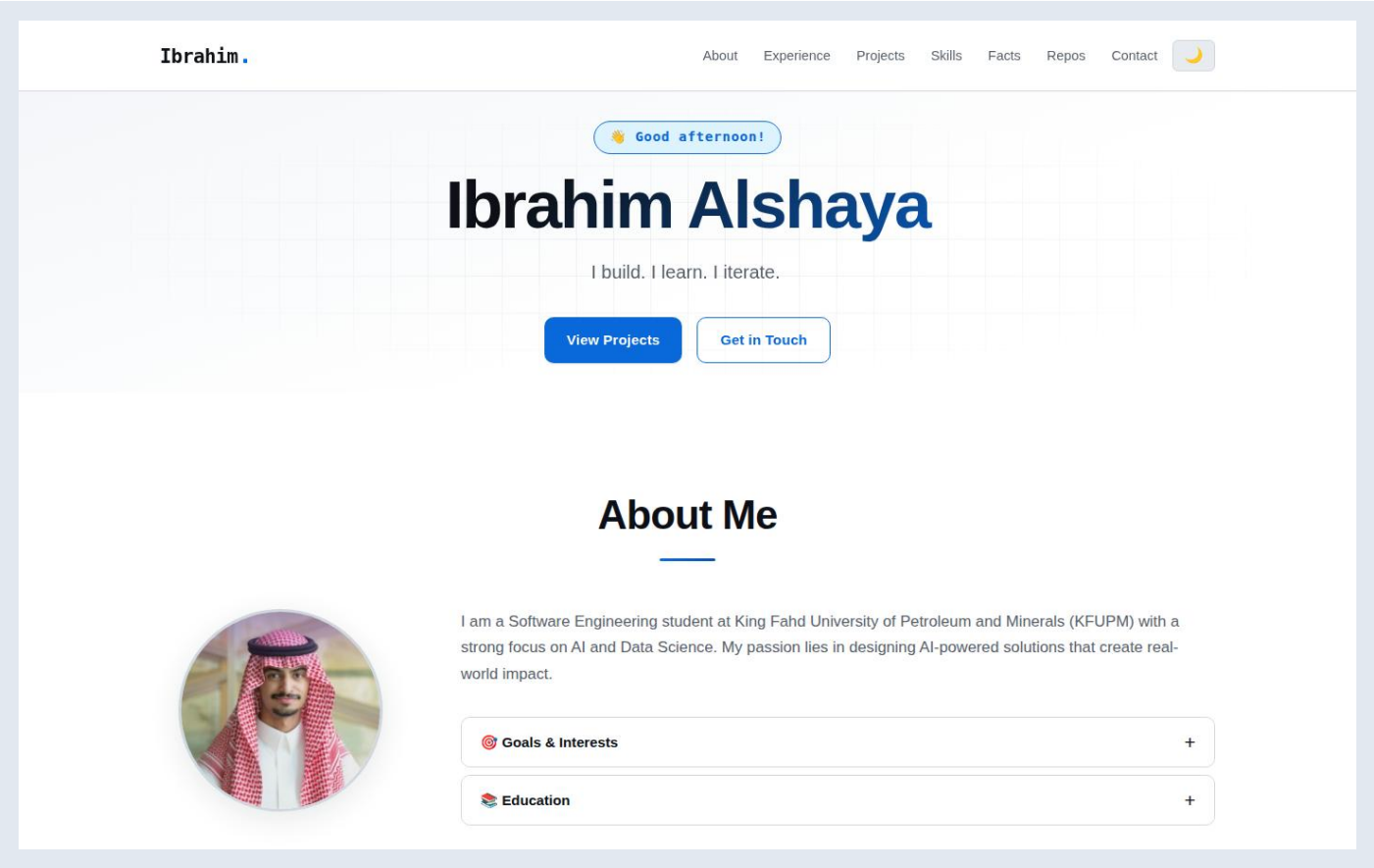
GitHub REST API for real repositories, with graceful fallbacks.

Manage client-side state honestly

Theme, section toggles, and filters persisted via localStorage.

DEMONSTRATION

https://ibrahimalshayea.netlify.app/



Dynamic project filtering

JSON-driven cards, category filters, Show More pagination.

Live GitHub integration

Repos fetched on load; sort by updated, stars, or name.

Theme + section state

Dark/light mode and collapsed sections survive reloads.

Responsive by default

Mobile-first CSS, 320px → 1920px+, hamburger nav.

Accessible form validation

Inline errors, loading state, minimum-length checks.

live walkthrough of the deployed site

HOW IT'S BUILT

No framework. No build step.

UI

Browser runtime

What the user sees

Semantic HTML5 · CSS Grid + Flexbox · CSS variables + dark theme

JS

Application logic

18 numbered modules in script.js

State via localStorage · IntersectionObserver scroll reveals · Client-side filter, sort, paginate

API

External data

Live fetches, with fallbacks

GitHub REST API — repositories · Useless Facts API -tech facts · Graceful offline fallback

0

DEPENDENCIES

No npm install. No bundler.

18

JS MODULES

One script, clearly numbered.

2

LIVE APIS

GitHub + Useless Facts.

WHAT WAS HARD

The honest list of challenges.

RESOLVED

Shipped in the app

- GitHub API with try/catch, retry button, and no partial renders on failure.
- Persistent state for theme and section toggles via localStorage, restored on load.
- Client-side filter + sort for repos without extra API calls - O(n log n).

NOT RESOLVED

Known limits today

- GitHub API is unauthenticated (60 req/hr); no caching layer yet.
- No search within fetched repos — only a language filter.

FUTURE WORK

Next iteration

- Email relay for the contact form (serverless function).
- Per-project detail pages and a small blog section.
- Add backend

HOW I USED AI

As a pattern assistant, not an autopilot.

AI ASSISTED WITH

- Boilerplate for fetch + try/catch patterns.
- Regex starters for email validation.
- Array.sort / filter scaffolds I then customised.
- Documentation structure and phrasing passes.

WHAT I OWNED

- All design decisions: palette, typography, layout, animations.
- Every line was read, tested, and modified to fit the project.
- Error messaging, UX states, accessibility labels — written by me.
- The project structure, JSON schema, and numbered module layout.

Claude — architecture, debugging, docs

GitHub Copilot — inline completions

ChatGPT — CSS animation review

CONCLUSION

A portfolio I'm happy to *actually link to.*

< 2s	90+	WCAG 2.1 AA
LOAD TIME ON 4G	LIGHTHOUSE MOBILE	ACCESSIBILITY TARGET

WHERE THIS GOES NEXT

Auth for the GitHub API, a serverless email relay for the contact form, PWA / offline support, and per-project case studies for the AI work.

Thank you.

github.com/ibshaya